Universidad Siglo 21

Licenciatura en Informática

Seminario de Práctica de Analista de Sistemas de Información

Sistema local de gestión de órdenes de trabajo para mantenimiento de líneas de 132 kV

Actividad práctica 4
Informe integrador del prototipo operacional Java Swing y MySQL

Alumno: Ramirez Luis Ignacio
Legajo: VINF04473
Docente: Pablo Alejandro Virgolini
Fecha de revisión: 8 de noviembre de 2026

Propósito y resultado de esta entrega

El proyecto aborda el registro y la consulta de órdenes de trabajo del mantenimiento de líneas de alta tensión. Esta versión integra un prototipo local de escritorio, organiza sus responsabilidades mediante un DAO y documenta la evidencia de ejecución. Se presenta un recorte operacional: alta, consulta, cambio de estado y guardado y recuperación de órdenes en archivo.

El código compiló con Java 17. Se ejecutaron 23 comprobaciones aprobadas, incluidas operaciones reales con MySQL 8.0.46 y Connector/J 8.4.0. Las capturas se obtuvieron de la interfaz Swing ejecutada en una pantalla virtual; se utilizaron exclusivamente datos sintéticos.

Se comprobó la ejecución desde IntelliJ IDEA en el equipo Windows del alumno, con consulta de la orden ID 1 en MySQL. La publicación del repositorio GitHub sigue pendiente. El video adjunto utiliza la grabación de voz aportada por Luis Ignacio Ramírez y subtítulos sincronizados; su duración es de 3 minutos y 21 segundos. Este informe no acredita participación en foros ni actividades académicas que no hayan sido documentadas.

Formato: A4 y espaciado simple. La versión editable está configurada en Calibri 11; el PDF generado en este entorno utiliza Carlito 11 como sustitución por falta de Calibri. Antes de enviar, exportar desde Word con Calibri disponible y completar el enlace del repositorio.

1 Modelo de negocio y justificación

La organización de referencia es ENERSA, en el sector de Mantenimiento de Líneas de Alta Tensión 132 kV, Zona Oeste. El documento de las actividades anteriores describe un circuito apoyado en órdenes en papel y transcripción posterior. Se retoma ese contexto aportado por el alumno; esta actividad no afirma haber realizado nuevas entrevistas ni mediciones de campo.

El proceso que se busca apoyar comienza cuando se identifica una tarea de mantenimiento, continúa con la emisión de una orden y su ejecución, y termina con el registro del resultado. En el circuito manual, consultar una intervención exige localizar sus registros. La propuesta permite almacenar y recuperar un conjunto mínimo de datos desde una computadora local.

Elemento	Definición para el proyecto
Usuario del prototipo	Personal de oficina que registra y consulta órdenes. No existe autenticación ni asignación de permisos en esta versión.
Información de entrada	Línea, piquete, descripción y tipo de mantenimiento.
Operaciones	Crear, consultar por ID, listar, cambiar estado y guardar o recuperar archivos.
Información de salida	Tabla de órdenes con ID, línea, piquete, descripción, tipo y estado.
Valor esperado	Facilitar la localización de registros y disminuir la transcripción repetida. No se midió una mejora real de tiempos ni del servicio eléctrico.

Alcance y restricciones

El ID autogenerado identifica el registro del prototipo; no representa el número de guía de una orden institucional. Se permiten los estados PENDIENTE, EN_PROCESO y FINALIZADA sin reglas de transición adicionales. No se incluyen gestión de usuarios, licencias eléctricas, activos asociados, historial completo ni aprobación de maniobras. El programa no sustituye los procedimientos operativos del sector.

Aplicación incremental del PUD

Inicio: se recuperan el problema, los interesados y el límite de las actividades previas. Elaboración: se ajusta la arquitectura a Java local, se define el contrato DAO y se concilia el esquema mínimo. Construcción: se implementan entidades, servicios, JDBC, interfaz y archivos. Transición: se prepara la entrega y la demostración. Se incorporó evidencia de ejecución desde IntelliJ en el equipo del alumno; la publicación y la revisión final forman parte del cierre pendiente. Estas fases organizan el trabajo y sus riesgos; no constituyen por sí solas evidencia de escalabilidad o eficiencia.

2 Trazabilidad de requisitos y correcciones

Requisito de la consigna	Implementación y evidencia
Java y aplicación local	Main inicia Swing; no hay aplicación móvil ni servidor REST.
Patrón coherente	DAO: OrdenDao y MySqlOrdenDao separan los servicios del SQL. Diagrama y justificación en sección 3.
Paquete de persistencia	lat.persistencia contiene Conexion, OrdenDao y MySqlOrdenDao.
Conectar y persistir en MySQL	Conexion.abrir e INSERT parametrizado; prueba real aprobada.
Consultar y actualizar	listar y actualizarEstado; tabla Swing, búsqueda por ID y cambio de estado.
Excepciones	SQLException propagada desde DAO; validaciones de entrada; avisos en Swing y registro técnico.
Abstracción o interfaz	Orden abstracta y contrato OrdenDao; Preventiva y Correctiva implementan getTipo.
Arreglos y ArrayList	String[] para columnas, tipos y estados; Object[] para filas; ArrayList<Orden> para resultados variables.
Archivos opcionales	ArchivoOrdenes escribe y recupera formato binario versionado; importación transaccional.
Correcciones anteriores	Clases en archivos y paquetes; comunicación local; una tabla real diferenciada de siete entidades de diseño.
Entrega y video	Fuentes, SQL, JAR, instrucciones, pruebas, capturas y presentación con voz del alumno y subtítulos, de 3 minutos y 21 segundos.
IDE y GitHub	Proyecto abierto y ejecutado desde IntelliJ IDEA en Windows; ver sección 15. El enlace publicado en GitHub sigue pendiente.

Las afirmaciones de pruebas de las entregas anteriores se reemplazan, para AP4, por los resultados reproducibles de evidencias/resultados-pruebas.txt. La evidencia actual no valida automáticamente todos los requisitos del diseño ampliado de las actividades anteriores.

3 Arquitectura local y patrón seleccionado

Se selecciona el patrón Data Access Object, DAO, porque el acceso JDBC necesita mantenerse separado de la presentación y de las validaciones. OrdenServicio depende de la interfaz OrdenDao; MySqlOrdenDao implementa las operaciones SQL. Esta estructura sigue la separación de acceso a datos descrita por Oracle (s. f.-a) y se aplica aquí a una aplicación local, sin adoptar los componentes de servidor de sus ejemplos históricos.

VentanaPrincipal captura y muestra datos. OrdenServicio valida y coordina. El modelo conserva los datos y sus invariantes. ArchivoOrdenes realiza la entrada y salida de archivos. Main construye estas dependencias de forma explícita. No se usa una conexión global única: cada operación abre y cierra su conexión.

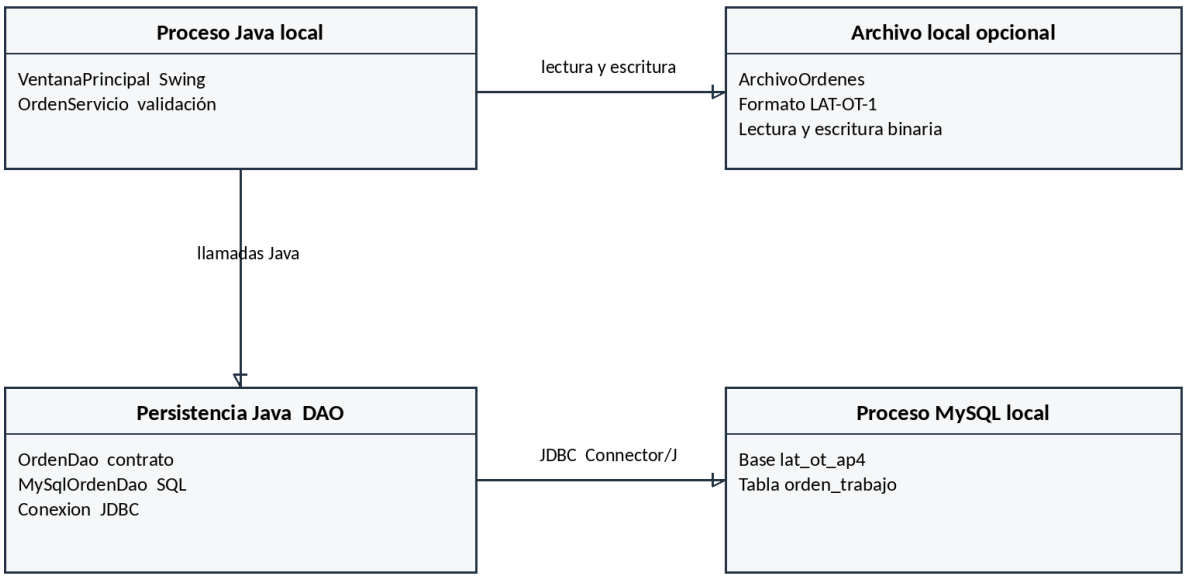


Figura 1. Despliegue base y dependencias. Elaboración propia a partir del código AP4.

La computadora ejecuta dos procesos: la JVM y MySQL. Las clases Java se comunican mediante llamadas dentro de la JVM. JDBC y Connector/J enlazan el DAO con el servidor MySQL de localhost. No hay HTTP, REST, interfaz web ni sincronización offline. La negociación TLS que pueda realizar JDBC con MySQL es distinta de una capa HTTPS, y no se afirma que toda configuración externa esté validada.

Las operaciones de entrada y salida se ejecutan en SwingWorker; done devuelve el resultado al hilo de eventos. Se deshabilitan las acciones durante cada operación. Esto evita ejecutar consultas JDBC en el hilo de la interfaz; no equivale a una prueba de carga ni a una solución multiusuario validada.

4 Clases principales y relaciones

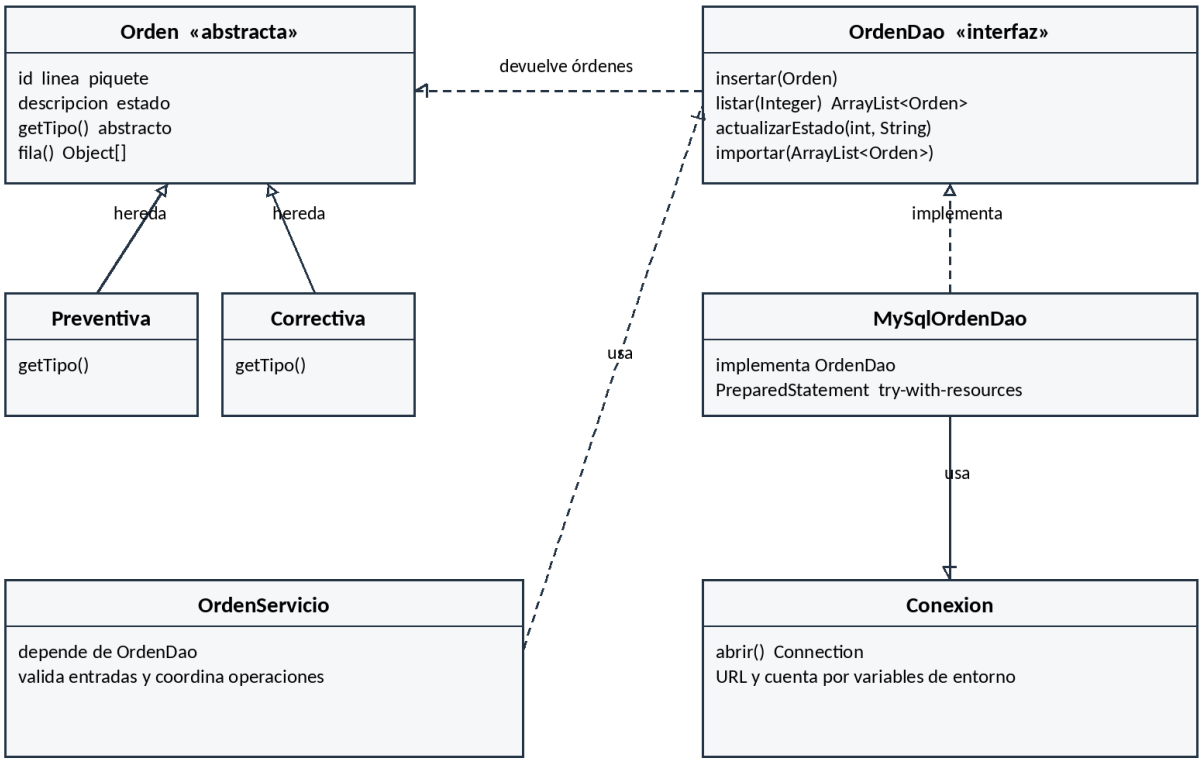


Figura 2. Vista simplificada de clases. Las flechas de herencia e implementación apuntan a la abstracción; las dependencias muestran el uso de un contrato o tipo.

Orden valida campos obligatorios y sus longitudes. Preventiva y Correctiva reutilizan esa validación y distinguen su tipo mediante getTipo. La abstracción evita instanciar una orden sin un tipo concreto. El método estático crear selecciona una de las dos variantes; no se lo presenta como un patrón adicional independiente.

OrdenDao define el contrato; MySqlOrdenDao lo implementa; Conexion encapsula URL, cuenta y apertura JDBC. OrdenServicio valida filtros y estados. VentanaPrincipal contiene formularios y acciones. ArchivoOrdenes guarda y recupera datos. Main constituye el punto de entrada. Los diez archivos de producción están separados en cinco paquetes: lat, lat.modelo, lat.persistencia, lat.servicio y lat.vista.

5 Casos de uso y secuencia de alta

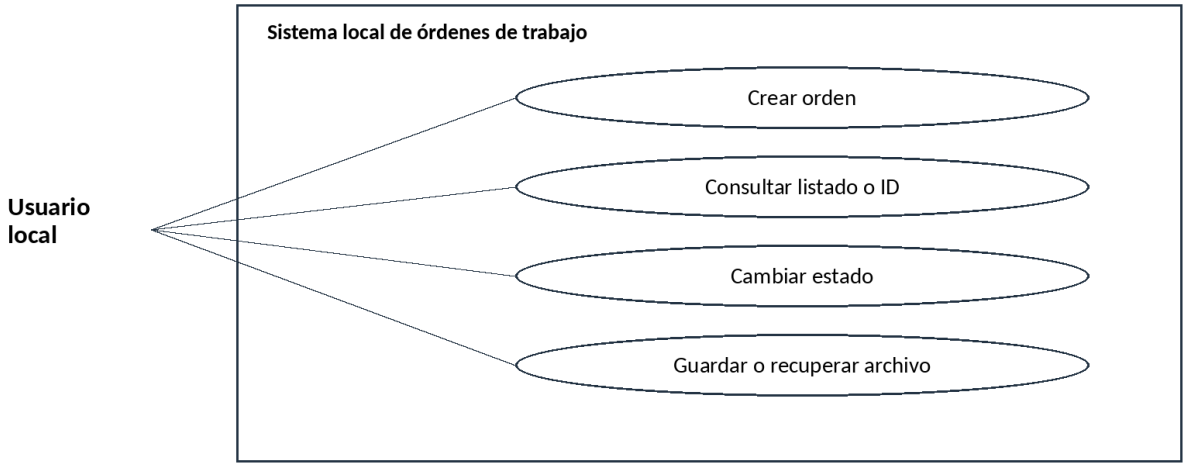


Figura 3. Casos de uso del prototipo. El actor representa a una persona que utiliza el escritorio local; no implica un rol autenticado.

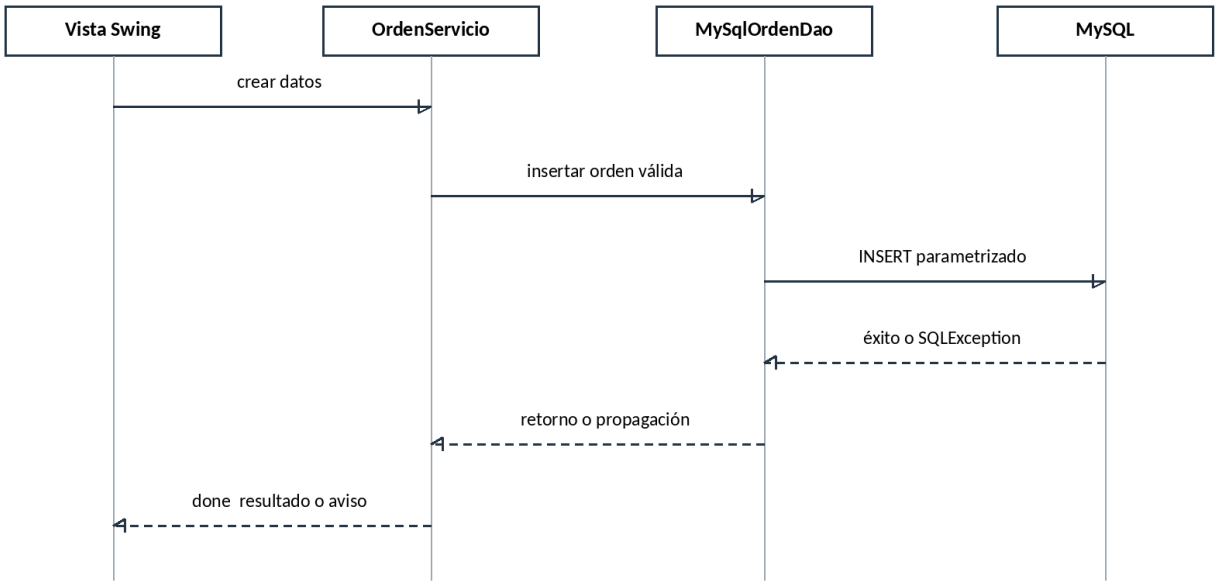


Figura 4. Secuencia simplificada de alta. La llamada desde la vista se ejecuta en el worker; el resultado se presenta en done. Ante datos inválidos, el servicio termina antes del INSERT. Ante SQLException, el DAO libera recursos y propaga la excepción hacia la interfaz.

Precondición de alta: base creada y cuenta configurada. Flujo principal: ingresar datos, aceptar, validar, insertar y confirmar. Alternativas: cancelar no modifica datos; campos inválidos impiden el alta; error de conexión informa que no se completó la operación. La consulta posterior permite comprobar la fila persistida.

6 Modelo de datos implementado

El prototipo AP4 utiliza la base lat_ot_ap4 y una tabla InnoDB, orden_trabajo. Esta elección se corresponde con el recorte operacional permitido por la consigna, que admite un modelo con algunas características del sistema final (Kendall y Kendall, 2011). La tabla carece de claves foráneas porque línea y piquete se almacenan como texto; no se presenta un DER con relaciones inexistentes.

Campo	Tipo SQL	Restricción o finalidad
id	INT	PK, AUTO_INCREMENT
linea	VARCHAR(100)	NOT NULL; texto no vacío
piquete	VARCHAR(30)	NOT NULL; texto no vacío
descripcion	VARCHAR(500)	NOT NULL; texto no vacío
tipo	ENUM	PREVENTIVA o CORRECTIVA; NOT NULL
estado	ENUM	PENDIENTE, EN_PROCESO o FINALIZADA; valor inicial PENDIENTE
creada_en	TIMESTAMP	NOT NULL; CURRENT_TIMESTAMP

Representación relacional: ORDEN_TRABAJO contiene siete atributos y una clave primaria, id. No hay relaciones entre tablas en el alcance implementado. El índice idx_linea_piquete facilita una futura búsqueda combinada, aunque la interfaz actual solo filtra por ID.

El diseño ampliado de las actividades anteriores contiene siete entidades distintas: zonas, lineas, piquetes, activos, usuarios, ordenes_trabajo y detalle_ot_activos. Los bloques SQL repetidos no representan 28 tablas diferentes. Ese diseño permanece como evolución futura y no es el esquema ejecutado por AP4.

Sentencias y decisiones

Alta: INSERT INTO orden_trabajo(linea,piquete,descripcion,tipo,estado) VALUES(?,?,?,?,?). Consulta: SELECT id,linea,piquete,descripcion,tipo,estado FROM orden_trabajo, con WHERE id=? cuando corresponde. Actualización: UPDATE orden_trabajo SET estado=? WHERE id=?. Los valores se asignan con PreparedStatement; el ID de búsqueda nunca se concatena como texto SQL.

El esquema se entrega en sql/esquema.sql. Se probó en MySQL 8.0.46; las restricciones CHECK requieren MySQL 8.0.16 o superior. La cuenta de aplicación necesita SELECT, INSERT y UPDATE; no requiere DELETE ni permisos de administración.

7 Arreglos colecciones y excepciones

Uso complementario de estructuras

La vista declara e inicializa String[] columnas con seis encabezados, String[] tipos con las dos variantes de mantenimiento y Orden.ESTADOS con tres valores. Se recorren los estados para validar selecciones. Orden.fila devuelve Object[] con seis posiciones y la vista lo incorpora al modelo de la tabla. Son estructuras de tamaño fijo porque su cantidad de elementos es conocida.

MySqlOrdenDao.listar crea ArrayList<Orden>, agrega una entidad por cada fila del ResultSet y devuelve un conjunto cuyo tamaño depende de la consulta. ArchivoOrdenes usa otro ArrayList para recuperar un lote. VentanaPrincipal recorre la lista y transforma cada entidad en un Object[] de presentación. Así, los arreglos definen la forma de cada fila y el ArrayList representa una cantidad variable de órdenes (Oracle, s. f.-b).

Responsabilidades frente a errores

Origen	Tratamiento y resultado
Campos vacíos, longitud, tipo o estado	Orden lanza IllegalArgumentException; la vista presenta el motivo sin registrar éxito.
ID no numérico, cero o desbordado	OrdenServicio rechaza la búsqueda antes de consultar MySQL.
Conexión, SQL o restricciones	El DAO declara SQLException; SwingWorker la transmite mediante ExecutionException; done muestra un aviso y el registro técnico conserva la excepción.
Lectura, formato o escritura de archivo	ArchivoOrdenes declara IOException; la vista informa el problema y no inicia la importación si la lectura no terminó.
Fallo dentro de un lote	MySqlOrdenDao.importar hace rollback; un error del rollback se agrega como excepción suprimida.

Las conexiones, sentencias y resultados se cierran mediante try-with-resources. En una inserción o actualización individual se utiliza autocommit. En la importación se desactiva y se confirma el lote con commit únicamente al terminar. El tratamiento de SQLException sigue la orientación de JDBC de conservar información diagnóstica y manejar el fallo en el nivel que puede informar al usuario (Oracle, s. f.-c).

8 Menú principal y práctica con archivos

Opción	Comportamiento observable
Nueva orden	Abre un formulario; guarda preventiva o correctiva con estado PENDIENTE.
Consultar	Filtro vacío: lista completa. ID entero positivo: busca el registro correspondiente.
Cambiar estado	Requiere una fila seleccionada; actualiza uno de los tres estados admitidos.
Guardar archivo	Consulta todas las órdenes y escribe un archivo .lat. Pide confirmar si el destino existe.
Recuperar archivo	Lee y valida el archivo completo; confirma con el usuario e inserta el lote en MySQL.
Salir	Cierra la ventana. Se requiere finalizar la operación activa antes de utilizar el menú.

Los botones de alta, consulta y cambio de estado comparten sus acciones con el menú. El resultado de la consulta se muestra en JTable, sin edición directa de celdas. Tras guardar o modificar se informa el resultado; Consultar actualiza el listado visible.

Formato y alcance del archivo

El archivo es binario: cabecera LAT-OT-1, cantidad de registros y cinco textos por orden: línea, piquete, descripción, tipo y estado. Se usan DataOutputStream y DataInputStream. El límite es de 10.000 órdenes y la lectura rechaza archivos mayores de 32 MB, cantidades inválidas, datos truncados y contenido adicional inesperado. No se deserializan objetos Java.

La escritura se realiza primero en un archivo temporal y después se reemplaza el destino. La recuperación asigna ID nuevos y genera una nueva fecha de creación. Repetirla produce duplicados, advertidos antes de confirmar. Esta función es un intercambio de datos del prototipo; no es una copia completa de seguridad de MySQL ni conserva auditoría histórica.

El alta y la importación son operaciones distintas: Nueva orden inicia siempre en PENDIENTE; recuperar conserva el estado del archivo. La validación de los estados se aplica en el modelo. Las pruebas comprueban la recuperación de caracteres Unicode, el rechazo de un archivo truncado y la inserción del lote en MySQL.

9 Evidencia del menú y el formulario

Capturas reales del programa ejecutado en una pantalla virtual de 1280 × 800. Se utilizan datos de demostración, sin información operativa real. Las imágenes originales se incluyen en evidencias para permitir ampliarlas.

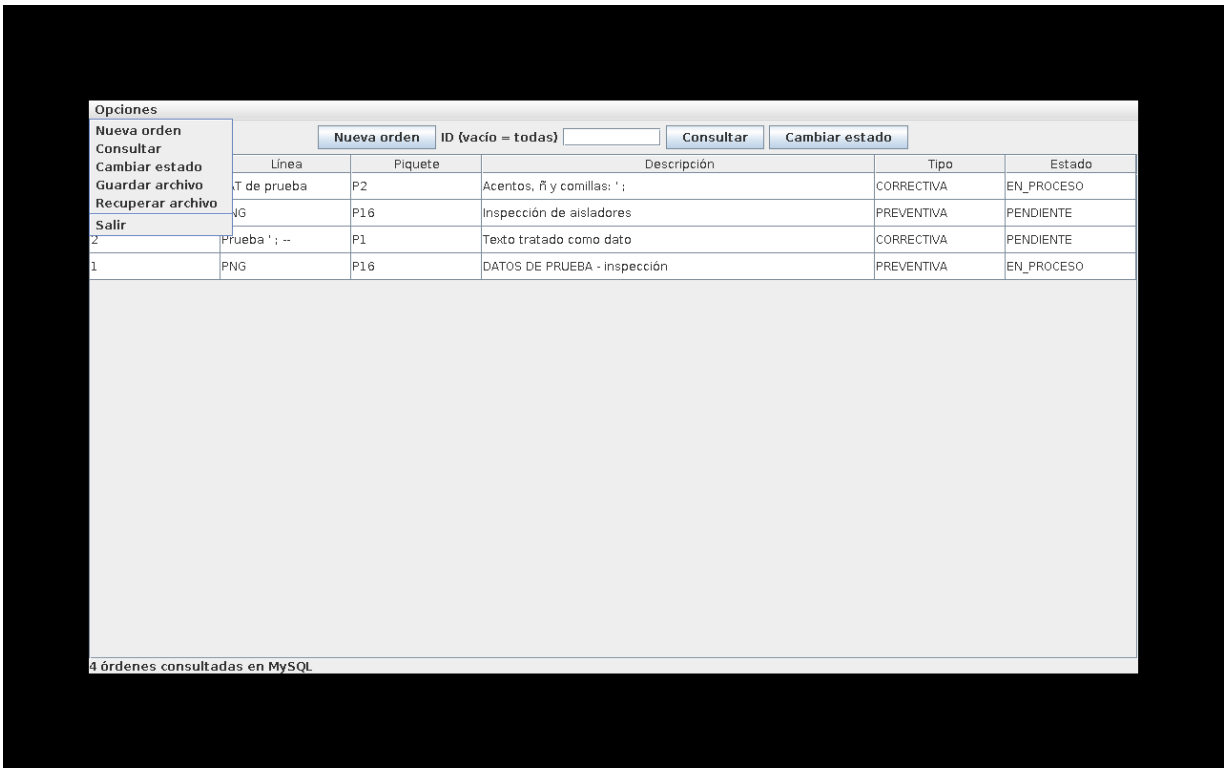


Figura 5. Menú Opciones desplegado y acciones disponibles.

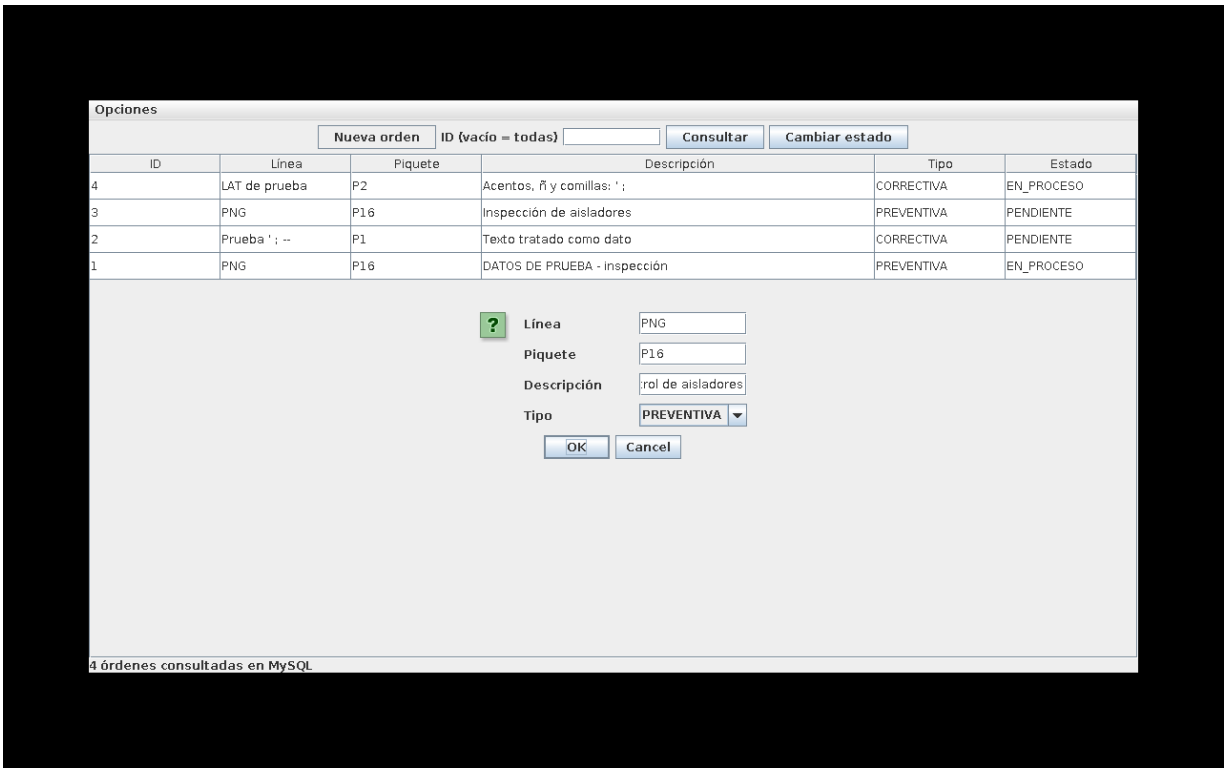


Figura 6. Formulario con línea PNG, piquete P16, descripción de prueba y tipo preventiva.

10 Evidencia de persistencia y búsqueda

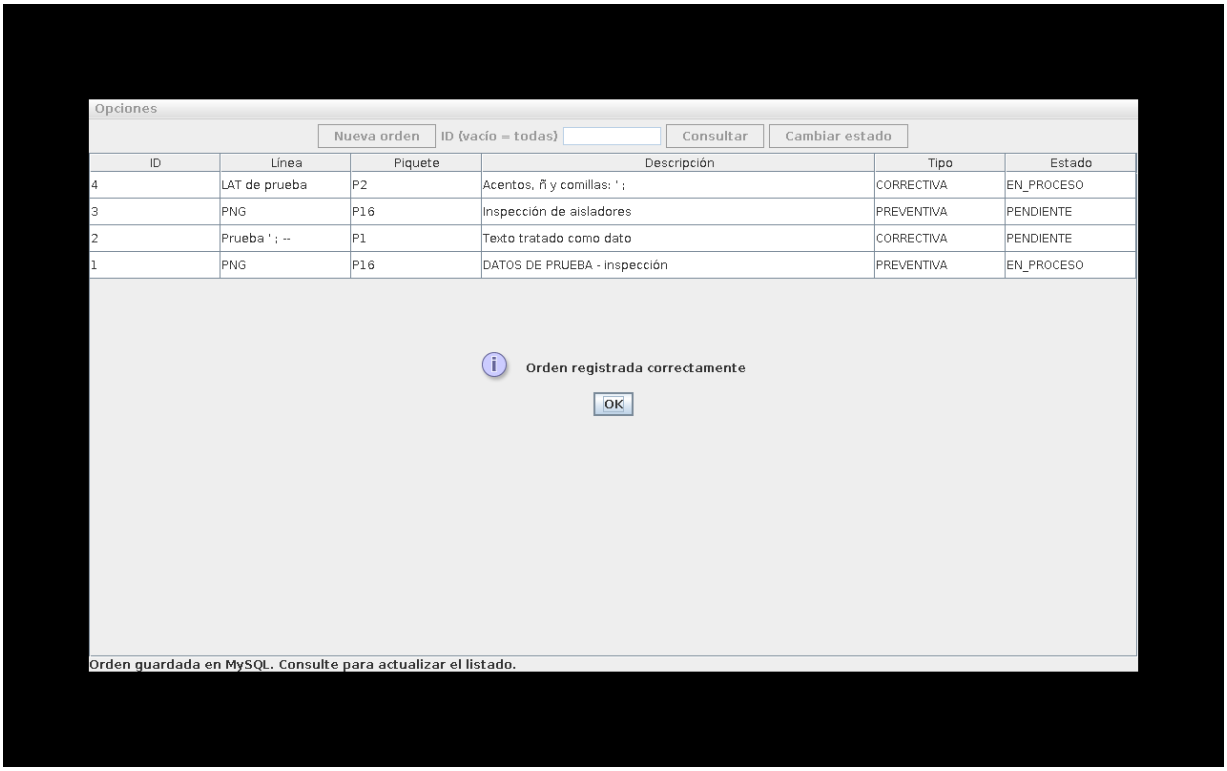


Figura 7. Confirmación del alta después de terminar el INSERT.

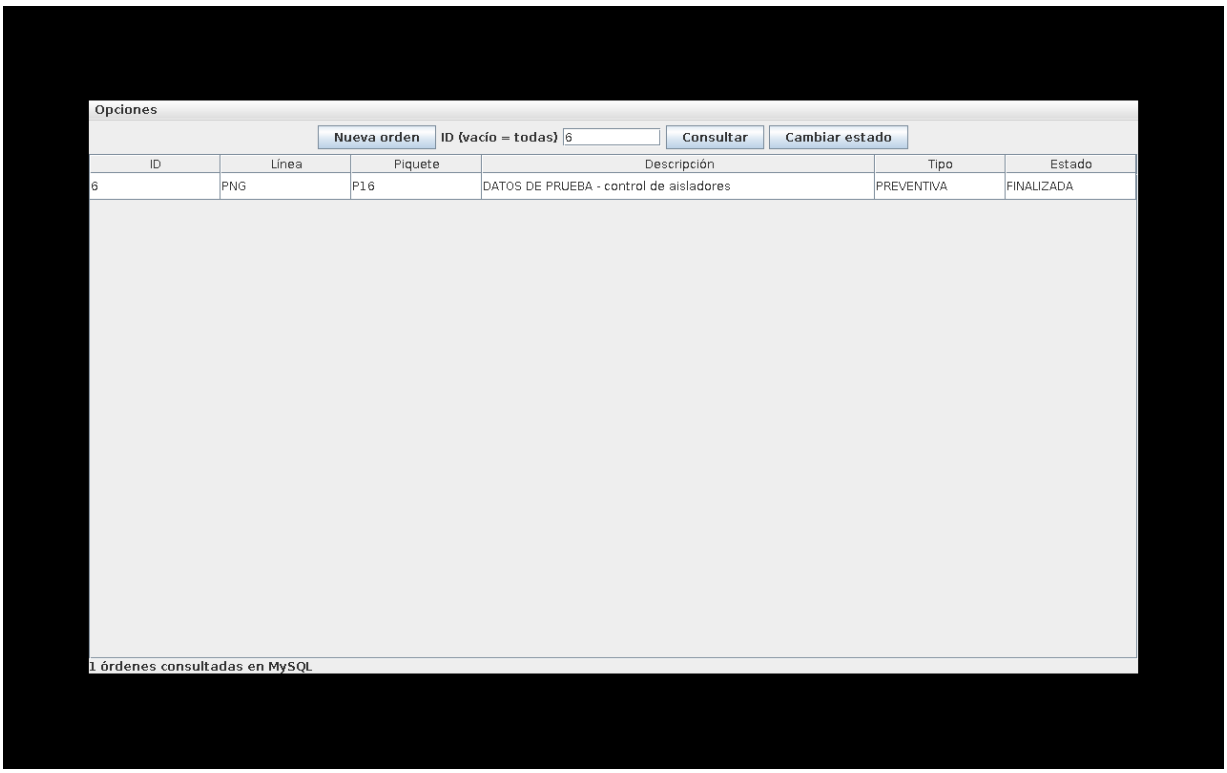


Figura 8. Consulta por ID. La única fila visible procede de MySQL; la consulta no es una búsqueda sobre una lista ficticia de la interfaz.

11 Evidencia de actualización y excepción

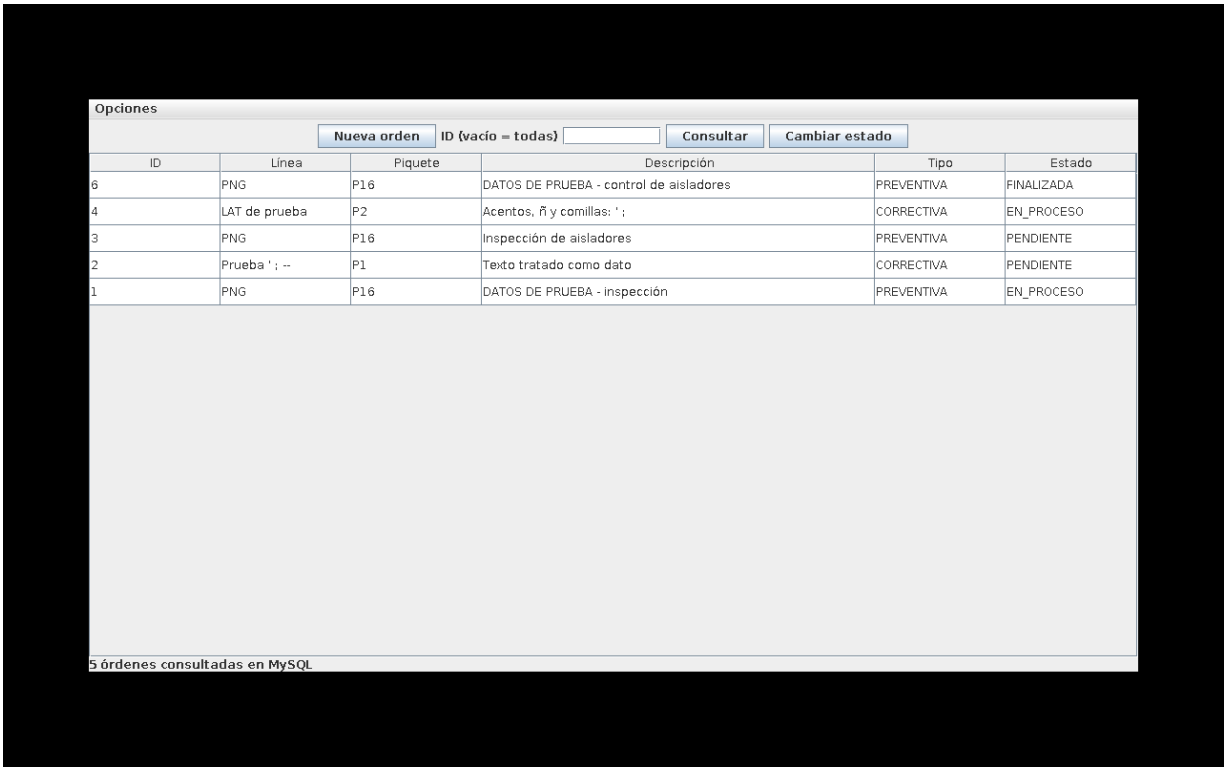


Figura 9. La orden creada en la demostración aparece con estado FINALIZADA después de actualizar y consultar. El archivo verificacion-mysql.txt confirma el mismo estado en la base.

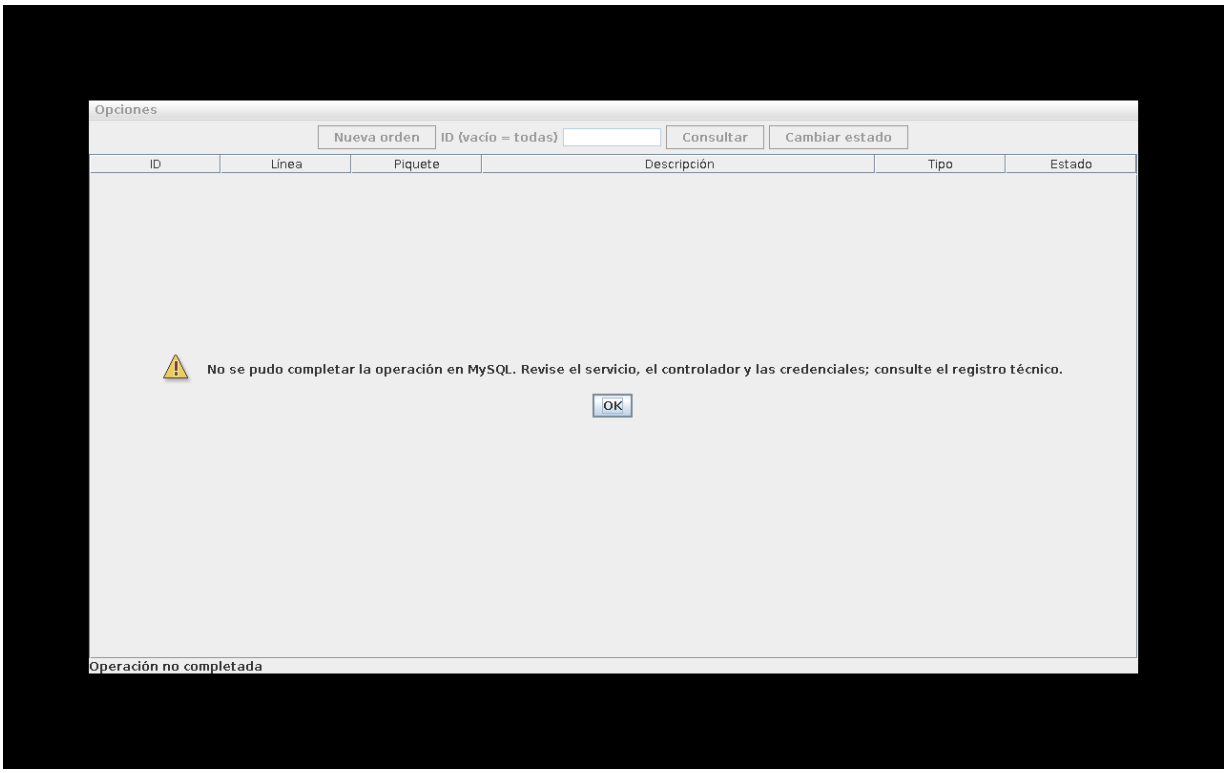


Figura 10. Aviso controlado ante una conexión deliberadamente inaccesible. Se verificó SQLException usando un puerto sin servicio; no se afirma haber simulado un corte de red durante una transacción.

12 Pruebas ejecutadas y límites

La clase Pruebas se ejecutó el 5 de noviembre de 2026 en un entorno separado de la PC del alumno. Resultado: 23 comprobaciones aprobadas, cero fallidas. Java 17.0.20, MySQL 8.0.46, Connector/J 8.4.0 y tabla InnoDB real. Los archivos de evidencias conservan la salida del test, la verificación SQL y las capturas. Los IDs omitidos en la secuencia son posibles tras rollback de una inserción con AUTO_INCREMENT.

Grupo	Casos y técnica	Resultado
Modelo y abstracción	Preventiva y correctiva; campos vacíos; tipos inválidos; valores límite de 100 y 101 caracteres.	6 aprobados
Arreglos	Fila de seis posiciones y catálogo de tres estados.	1 aprobado
Archivos	Escritura y lectura Unicode; rechazo de archivo truncado.	2 aprobados
Filtro	Texto no numérico y desbordamiento del entero.	2 aprobados
Integración MySQL	Alta/listado, búsqueda por ID, actualización, persistencia en nueva conexión, ID inexistente, modificación inexistente, estado inválido y texto con caracteres SQL.	8 aprobados
Lotes	Recuperación de archivo en MySQL; error SQL propagado; rollback revierte el primer INSERT del lote.	3 aprobados
Conexión	Destino inaccesible produce SQLException controlable.	1 aprobado

El recorrido Capturas ejecutó además el menú, el formulario, el alta, la actualización, la búsqueda y los avisos en Swing. Es evidencia de demostración, separada del conteo de las 23 comprobaciones. La actualización se contrastó con una consulta SQL posterior.

La ejecución en Windows e IntelliJ se documenta en la sección 15. No se probaron concurrencia, grandes volúmenes, rendimiento, corte durante commit, recuperación ante caída del equipo, accesibilidad. Tampoco se acredita aceptación de usuarios ni mejoras de tiempos en campo. La autenticación, los permisos y la normalización del diseño ampliado quedan fuera del prototipo.

13 Instalación entrega y presentación

La PC del trabajo no requiere instalaciones para revisar el PDF, las imágenes o el video. La ejecución del sistema requiere un equipo autorizado con JDK 17 y MySQL. Crear la base con sql/esquema.sql, preparar una cuenta con permisos mínimos y configurar LAT_DB_URL, LAT_DB_USER y LAT_DB_PASSWORD. El README detalla el procedimiento. El puerto utilizado en las pruebas fue 13307; el valor base del proyecto es localhost:3306.

El proyecto se abrió como proyecto Maven en IntelliJ IDEA 2026.3, con Eclipse Temurin JDK 17.0.21.9 en la PC Windows del alumno. La evidencia muestra la configuración Main activa, Main.java abierto, el directorio target y el programa consultando MySQL. En ese equipo se instaló MySQL Community Server 8.4.13. Esta comprobación manual complementa las 23 pruebas automatizadas ejecutadas previamente en el entorno separado; no representa una nueva ejecución de esos 23 casos.

El JAR compilado se entrega sin incluir el controlador dentro de él. En Windows se ejecuta con java -cp "lat-ap4.jar;lib/*" lat.Main; en Linux, con java -cp "lat-ap4.jar:lib/*" lat.Main. Connector/J 8.4.0 y su licencia se incluyen en el paquete. Los scripts compilar.bat y compilar.sh permiten reconstruir el JAR a partir de las fuentes.

Enlace de GitHub pendiente de publicación

Todavía no hay un repositorio publicado y verificado del alumno. Debe incorporarse aquí su URL real antes de presentar la actividad. El enlace genérico github.com no satisface este requisito. Publicar el código, SQL, pom.xml, instrucciones, diagramas y evidencia de prueba; excluir claves, bases de datos, .env y directorios de compilación. Esta tarea no puede cerrarse sin acceso a la cuenta del alumno.

Video de presentación con voz del alumno

El video acompaña este informe con la voz del alumno, subtítulos integrados y capturas reales del prototipo. La presentación dura 3 minutos y 21 segundos, conservando el ritmo de la grabación aportada. Los cambios de escena acompañan la explicación del problema, el diseño, las funciones y las pruebas. Se adjunta además el archivo SRT para editar o reutilizar los subtítulos.

Tiempo	Contenido
0:00 a 1:00	Presentación, problema, PUD y alcance operacional.
1:00 a 1:37	Arquitectura, DAO y clases abstractas.
1:37 a 2:13	Alta, confirmación, búsqueda y cambio de estado.
2:13 a 3:21	Arreglos, ArrayList, archivos, excepciones, pruebas y futuras ampliaciones.

14 Referencias y revisión previa al envío

Kendall, K. E., & Kendall, J. E. (2011). Análisis y diseño de sistemas. Pearson Education. Referencia provista por la consigna; se utiliza el concepto de prototipo operacional allí citado.

Oracle. (s. f.-a). Core J2EE patterns: Data access object. https://www.oracle.com/java/technologies/dataaccessobject.html

Oracle. (s. f.-b). ArrayList. Java Platform, Standard Edition 17 API specification. https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/ArrayList.html

Oracle. (s. f.-c). Handling SQLExceptions. The Java Tutorials. https://docs.oracle.com/javase/tutorial/jdbc/basics/sqlexception.html

Oracle. (s. f.-d). MySQL Connector/J developer guide. https://dev.mysql.com/doc/connector-j/en/

Ramirez, L. I. (2026). Sistema local de gestión de órdenes de trabajo para mantenimiento de líneas de alta tensión 132 kV [Documento de trabajo de las actividades 1, 2 y 3, no publicado].

Las referencias técnicas sustentan las decisiones de diseño y programación. La prueba del funcionamiento de este prototipo procede de sus archivos de evidencia; las referencias externas no certifican el programa. Los diagramas son de elaboración propia y describen el código AP4.

Cierre de requisitos verificables

Completado en esta versión: fuente Java separada por responsabilidades, abstracción e interfaz, DAO justificado, conexión y operaciones reales en MySQL, interfaz Swing con menú, arreglos y ArrayList complementarios, excepciones, práctica de archivos, diagramas y evidencia de prueba. El modelo SQL presentado coincide con el esquema ejecutado.

Pendientes antes del envío: incorporar el enlace real de GitHub; exportar el editable con Calibri 11 donde la fuente ya esté disponible; realizar la revisión final de la presentación con voz del alumno y subtítulos. La participación en actividades comunes solo puede acreditarse con la evidencia real del alumno.

Se mantiene un alcance de prototipo. La consigna permite implementar algunas características del sistema final; esa autorización no elimina la obligación de demostrar que los módulos entregados funcionan ni de declarar los límites que aún existen.

15 Verificación en el equipo del alumno

Se incorporan capturas aportadas por Luis Ignacio Ramírez durante la instalación y prueba local. El recorrido documenta la creación de una orden preventiva, su consulta y la modificación de su estado. La captura siguiente muestra la ejecución desde IntelliJ y la consulta de la orden ID 1 con estado FINALIZADA. Los datos cargados son de prueba.

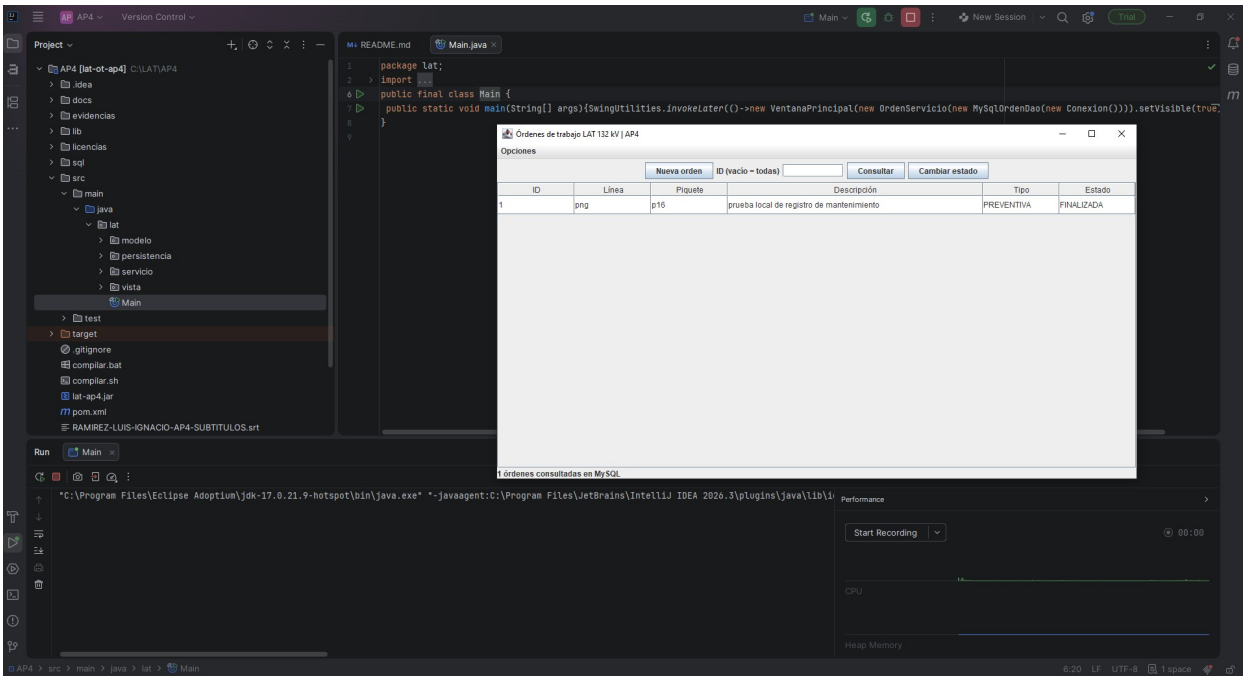


Figura 11. Proyecto abierto en IntelliJ IDEA con Main.java y la configuración Main activa; aplicación Swing mostrando un registro consultado en MySQL. Fuente: captura aportada por el alumno.

Entorno observado: Windows, IntelliJ IDEA 2026.3, Eclipse Temurin JDK 17.0.21.9 y MySQL Community Server 8.4.13. La versión del JDK y la IDE aparecen en la línea de ejecución de la captura; la versión de MySQL se observó en la consola del proceso de configuración local.

Las capturas windows-01-alta.png y windows-02-estado.png, incluidas en evidencias, muestran la misma orden inicialmente PENDIENTE y luego EN_PROCESO. La figura 11 muestra su consulta posterior como FINALIZADA. Esta evidencia manual se mantiene separada del informe automatizado de 23 comprobaciones. No se atribuyen resultados de carga, concurrencia o aceptación de usuarios que no se hayan realizado.